MetaFeature-Orchestrator:

Building Defensible, Scalable Evaluation for Generative AI

Jane Huang

02/2026

Evaluation Must Become Infrastructure

As generative AI systems move from isolated chat interfaces into deeply embedded product features, evaluation can no longer be treated as a downstream activity. It must become infrastructure.

Modern technology platforms now deploy hundreds of AI-powered capabilities — spanning writing assistance, visual generation, translation, summarization, contextual automation, and reasoning — across global user bases and dozens of languages.

These systems evolve rapidly and exhibit increasing autonomy. Under these conditions, traditional evaluation approaches — reliant on manually authored prompts, ad hoc human review, or loosely governed automated judges — no longer satisfy the standards required for quality assurance, safety governance, cross-team comparability, or regulatory defensibility.

What's missing is not more evaluators.

What's missing is a governed evaluation layer.

This article introduces the **MetaFeature-Orchestrator** — a metadata-driven evaluation infrastructure designed for large-scale generative AI platforms. Rather than treating evaluation prompts as disposable artifacts, the Orchestrator treats them as versioned contracts derived systematically from:

- Explicit feature metadata
- Canonical feature taxonomies
- A centralized metric registry
- Structured Responsible AI constraints

By encoding human evaluation standards into a deterministic orchestration layer, the system guarantees:

- Auditability
- Reproducibility
- Cross-feature comparability
- Structural enforcement of safety, privacy, and bias constraints

Responsible AI requirements — including hallucination detection, privacy protection, and bias assessment — are injected by construction and cannot be bypassed by individual teams.

Evaluation becomes governed. Deterministic. Defensible. And scalable.

1. Problem Statement and Strategic Context

1.1 The Limits of Manual and Ad-Hoc Evaluation

Early generative AI systems were evaluated using manually written prompts, qualitative human review, and feature-specific heuristics. These approaches were sufficient for experimentation or isolated use cases.

They do not scale to modern AI deployments characterized by:

- **Scale**: Hundreds of AI-enabled features across multiple product surfaces
- **Diversity**: Dozens of languages, cultural contexts, and regional norms
- **Velocity**: Rapid model iteration cycles

- **Risk**: Hallucination, privacy leakage, bias, and inappropriate tone

At this scale, evaluation must move upstream — from quality control to core infrastructure.

1.2 The Limits of LLM-as-Judge Without Prompt Governance

Automated evaluation using large language models as judges improves reviewer scalability, but it does not solve governance.

LLM-based judges are highly sensitive to:

- Prompt phrasing
- Omitted constraints
- Ambiguous scoring instructions

Without systematic prompt control:

- Evaluation criteria drift over time
- Safety checks are inconsistently applied
- Scores become incomparable across releases

The core problem is not a shortage of judges — human or automated.

It is the absence of a governed evaluation layer

2. Taxonomy as a Control Mechanism

Quality in generative AI is not universal. It is task-dependent, risk-dependent, and context-dependent.

A summarization system should not be evaluated using the same rubric as a creative image generator. A translation feature should not be judged by the same criteria as an autonomous task planner.

The MetaFeature-Orchestrator treats feature taxonomy as a governing abstraction.

Each canonical category defines:

- Applicable quality metrics
- Dominant failure modes
- Mandatory Responsible AI constraints
- Evaluation prompt structure

This ensures that fundamentally different behaviors are never evaluated using incompatible standards — even if they share similar interfaces or underlying models.

Importantly, metrics are **stable but not static**.
They evolve deliberately through versioning, preserving longitudinal comparability while allowing adaptation to emerging risks.

Metric definition follows a human-in-the-loop governance model: feature teams propose candidate metrics, while a centralized evaluation authority validates cross-feature consistency. AI assistance may suggest metrics, but final approval remains human.

Governance augments expertise — it does not replace it.

3 Case Study: A Modern Integrated AI Platform

Publicly observable AI ecosystems provide a useful illustration of evaluation challenges at scale.

Modern device- and OS-level AI platforms integrate generative capabilities directly into:

- Communication systems
- Writing tools
- Visual understanding
- Media processing
- Contextual assistants
- Health and coaching domains
- Developer APIs

These capabilities operate across devices, languages, and hybrid on-device/cloud environments.

Such integration introduces complex evaluation requirements:

- Cross-locale semantic fidelity
- Contextual grounding to personal data
- Consistency across heterogeneous inference environments
- Trajectory-level reasoning correctness

Evaluation cannot be feature-by-feature improvisation. It must be systemic.

4 Canonical Feature Categories

AI feature portfolio spans a wide range of generative and assistive behaviors— including summarization, translation, content generation, reasoning, and automation—each with distinct objectives, failure modes, and risk profiles. As these capabilities are deployed globally and exhibit increasing autonomy, evaluation cannot rely on a single, uniform standard. Instead, it requires a taxonomy-driven framework that explicitly defines what quality means for each class of behavior. The MetaFeature-Orchestrator operationalizes this principle by organizing features into a set of canonical categories. Each category serves as a governing abstraction that determines the applicable evaluation metrics, dominant risks, and required Responsible AI constraints. This ensures that features with fundamentally different intents are never evaluated using incompatible or overly generic criteria.

Category	**Representative Examples**	**Primary Evaluation Focus**
Summarization	Email summaries, article digests	Faithfulness, hallucination avoidance
Writing Assistance	Smart reply, tone rewriting	Relevance, tone, privacy
Translation	Real-time multilingual translation	Semantic equivalence
Assistant & Reasoning	Task planning, contextual workflows	Correctness, grounding
Creative Generation	Image & emoji generation	Safety, bias, policy compliance

A controlled custom category is provided for novel or emerging feature types. Features assigned to this category must explicitly declare their evaluation metrics and are subject to mandatory Responsible AI enforcement, ensuring that innovation does not bypass governance or safety standards.

5. System Architecture Overview

5.1 Design Principles

The MetaFeature-Orchestrator is designed as **evaluation infrastructure**, not an experimental agent. Its architecture is governed by a small set of non-negotiable principles that ensure reliability, scalability, and long-term governance:

- **Statelessness**
  The orchestration layer maintains no hidden or implicit memory. All state—feature definitions, metric configurations, prompt versions, and evaluation results—is persisted in externally governed stores. This simplifies reasoning about system behavior and enables horizontal scalability.

- **Auditability and Traceability**
  Every evaluation outcome is fully traceable to the exact prompt template, metric definitions, and version identifiers used to produce it. This guarantees reproducibility, supports regression analysis, and enables compliance review.
- **Structural Responsible AI Enforcement**
  Safety, privacy, and bias constraints are injected by construction rather than configured ad hoc. Responsible AI requirements are enforced uniformly across features and cannot be bypassed at the feature level.
- **Cost Awareness**
  The system is optimized to minimize unnecessary model invocation. Prompt generation is performed offline and deterministically wherever possible, reserving model calls for evaluation execution rather than orchestration logic.

Together, these principles ensure that evaluation remains consistent, defensible, and economically viable as the number of features and supported locales grows.

5.2 Dual-Mode Architecture

To balance determinism with extensibility, the MetaFeature-Orchestrator operates in two complementary modes that share a common execution pipeline and output schema:

- **Template Mode**
  A deterministic prompt writer generates canonical evaluation contracts from predefined, versioned templates. This mode is optimized for standard feature categories and provides maximum reproducibility, consistency, and operational efficiency.
- **Agentic Mode**
  A controlled reasoning agent is used for novel or complex features that cannot be fully resolved through static templates. The agent selects metrics, constraints, and templates through explicit, auditable tool invocation. Importantly, this mode remains deterministic at the prompt-generation layer, preserving reproducibility despite dynamic reasoning.

Both modes produce structurally identical evaluation prompts and feed into the same downstream evaluation and scoring pipeline. This separation allows the system to accommodate innovation and feature diversity without compromising governance or consistency.

6. Prompt Orchestration Methodology

6.1 Feature Metadata Schema

Each AI feature is onboarded through a structured metadata descriptor that serves as the **authoritative input** to the evaluation system. Rather than relying on manually crafted prompts, the MetaFeature-Orchestrator derives evaluation behavior directly from this metadata, ensuring consistency and governance at scale.
The metadata schema captures four essential dimensions:

- **Feature Domain**
  Identifies the canonical feature category and determines the applicable metric families and evaluation semantics.
- **Behavioral Intent**
  Explicitly defines the feature's success criteria, clarifying what constitutes correct or high-quality behavior.
- **Locale Context**
  Encodes language- and culture-specific constraints, enabling evaluation to account for regional norms, linguistic formality, and localized expectations.
- **Responsible AI (RAI) Constraints**
  Declares prohibited behaviors, elevated risk flags, and domain-specific safety requirements.

This metadata functions as the **single source of truth** for evaluation generation, eliminating ambiguity and preventing feature-specific divergence in evaluation standards.

6.2 Template Grammar

All evaluation prompts generated by the Orchestrator adhere to a strict, shared grammar. This grammar defines not only the structure of the prompt, but also the semantics of judgment, ensuring that evaluation results are reproducible and comparable across features and releases.

Each prompt is composed of the following elements:

- **Judge role and authority definition**
- **Grounding rules and explicit prohibitions** (e.g., no external knowledge)
- **Feature and locale context**
- **Metric definitions with explicit scoring instructions**
- **Standardized scoring scale**
- **Strict JSON output schema**
- **Evaluation inputs** (source content and model output)

Flexibility is intentionally constrained at this layer. Allowing ad hoc prompt variation introduces uncontrolled variance, undermines longitudinal analysis, and re-introduces evaluator drift. Rigid prompt grammar is therefore a prerequisite for reliable large-scale evaluation.

6.3 Responsible AI Injection

Responsible AI requirements—including privacy protection, bias mitigation, hallucination detection, and safety enforcement—are injected automatically based on the feature's category and declared risk profile. These constraints are applied structurally during prompt generation and **cannot be disabled or overridden** by individual feature teams.

By embedding Responsible AI enforcement directly into the orchestration layer, the system ensures uniform application of ethical and safety standards across the entire AI ecosystem.

7. Advanced Evaluation Techniques

7.1 Hallucination Detection via Claim Extraction

In high-stakes summarization or RAG workflows, a binary "Pass/Fail" score is insufficient. To provide actionable feedback, the **MetaFeature-Orchestrator** implements a granular, three-stage "Extract-Verify-Score" pipeline.

- **Atomic Claim Extraction**

The system first decomposes a generated summary into a list of **Atomic Claims**. A claim is considered atomic if it represents a single, verifiable fact that can stand alone.

- **Prompt Engineering Strategy:** The Orchestrator injects instructions to resolve anaphora (e.g., replacing "it" or "the company" with specific names) to ensure each claim is **decontextualized**.
- **Example:**
  - *Summary:* "The model was released in 2024 by Meta and supports 30 languages."
  - *Atomic Claim A:* "The model was released in 2024 by Meta."
  - *Atomic Claim B:* "The model supports 30 languages."

- **Verification via NLI (Natural Language Inference)**

Each claim is then cross-referenced against the source document (the "ground truth") using a secondary NLI-specialized model. For every claim, the Orchestrator expects one of three labels:

- **Entailment:** The source text explicitly supports the claim.
- **Contradiction:** The source text provides evidence that directly refutes the claim (Intrinsic Hallucination).
- **Neutral:** The source text neither supports nor refutes the claim (Extrinsic Hallucination).

- **Scoring & Traceability**

The final **Hallucination Score** is calculated as the ratio of supported claims to total claims. Crucially, the Orchestrator provides **provenance tags**—highlighting exactly which sentence in the source document supported a claim, or exactly which generated sentence contained a contradiction.

Technical Tip: In production, we found that using a "Chain-of-Verification" (CoVe) approach—where the model is first asked to extract claims and then independently verify them in separate steps—significantly reduces the "judge hallucination" effect compared to doing both in a single prompt.

7.2 Cross-Lingual Evaluation

To support global deployment, the system evaluates non-English outputs using cross-lingual semantic representations rather than relying on English back-translation. This design reduces English-centric bias, preserves local linguistic nuance, and enables more faithful assessment of meaning across languages.

8. Operational Workflow

The MetaFeature-Orchestrator operates as a closed-loop evaluation pipeline designed to support continuous quality assurance and long-term governance at scale:

- **Onboarding**
  Feature teams submit structured metadata descriptors that define feature intent, category, locale context, and Responsible AI constraints.
- **Prompt Synthesis**
  The Orchestrator generates governed, versioned evaluation prompts deterministically based on the submitted metadata and canonical templates.
- **Execution**
  Evaluation prompts are executed against model outputs to produce structured, machine-readable scores and diagnostics.
- **Validation**
  Targeted human review samples are used to calibrate automated judges and verify alignment with human quality standards.
- **Governance**
  Evaluation results, prompt versions, and metric configurations are persistently stored, versioned, and audited.

This closed-loop workflow enables reliable regression tracking, trend analysis across releases, and continuous monitoring of quality and risk over time.

9. Governance and Ownership

Evaluation standards within the MetaFeature-Orchestrator are centrally governed to ensure consistency and defensibility across the ecosystem. Metric definitions, prompt templates, and Responsible AI constraints are owned, reviewed, and versioned by a centralized evaluation authority.

Feature teams retain responsibility for providing accurate and complete metadata that reflects their feature's behavior and intent. However, they cannot override or bypass evaluation standards or Responsible AI requirements. This clear separation of responsibilities preserves product-level flexibility while maintaining organizational consistency, comparability, and accountability.

Conclusion: Evaluation as Strategic Infrastructure

The MetaFeature-Orchestrator transforms generative AI evaluation from an ad hoc, feature-specific activity into durable platform infrastructure.

By encoding judgment into governed systems, it enables innovation without sacrificing auditability, comparability, or ethical accountability.

As AI systems become increasingly agentic and embedded into critical workflows, evaluation must evolve beyond static prompt scoring toward:

- Metadata-driven orchestration
- Structural safety enforcement
- Trajectory-level reasoning assessment

In an era where models evolve weekly and AI agents begin to act autonomously, evaluation infrastructure may ultimately become more important than the models themselves.

Organizations that treat evaluation as infrastructure — not an afterthought — will define the next phase of responsible AI at scale.

References

- Zheng et al., *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena* (NeurIPS 2023)
  https://arxiv.org/abs/2306.05685
- Liu et al., *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment* (EMNLP 2023)
  https://arxiv.org/abs/2303.16634
- Huang et al., *A Survey on LLM-as-a-Judge* (2024)
  https://arxiv.org/abs/2411.15594
- Google Cloud, *Master Generative AI Evaluation: From Single Prompts to Complex Agents*
  https://cloud.google.com/blog/topics/developers-practitioners/master-generative-ai-evaluation-from-single-prompts-to-complex-agents
- Vertex AI Documentation, *Evaluate Gen AI Agents*
  https://cloud.google.com/vertex-ai/docs/generative-ai/models/evaluation-overview
- Dave Davies, *AI Agent Evaluation: Frameworks, Strategies, and Best Practices*
  https://medium.com/online-inference/ai-agent-evaluation-frameworks-strategies-and-best-practices-9dc3cfdf9890
- OpenReview, *Meta-Evaluation Collapse*
  https://openreview.net/forum?id=IF0L7HSs3K
- HaluEval: *A Hallucination Evaluation Benchmark for LLMs*
  https://github.com/RUCAIBox/HaluEval
- AWS Machine Learning Blog, *Detect Hallucinations for RAG-Based Systems*
  https://aws.amazon.com/blogs/machine-learning/detect-hallucinations-for-rag-based-systems/
- OpenAI Cookbook, *How to Evaluate Abstractive Summarization*
  https://cookbook.openai.com/examples/evaluation/how_to_eval_abstractive_summarization
- Prompting Guide, *Automatic Prompt Engineer (APE)*
  https://www.promptingguide.ai/techniques/ape
- Arize Phoenix, *LLM-as-a-Judge Prompt Optimization*
  https://arize.com/docs/phoenix/cookbook/prompt-engineering/llm-as-a-judge-prompt-optimization
- ACL 2025, *Cross-Lingual Auto Evaluation for Assessing Multilingual LLMs*
  https://aclanthology.org/2025.acl-long.1419/
- Nexla, *AI Metadata: Key Concepts & Best Practices*
  https://nexla.com/ai-readiness/ai-metadata/
- NIST, *AI Risk Management Framework (AI RMF)*
  https://www.nist.gov/itl/ai-risk-management-framework